



**UNIVERSIDADE
FEDERAL DE
OURO PRETO**

Faculdade de Ciência da Computação

Organização de Computadores Trabalho Prático II

Marcos Vinicio Euzébio
Luisa Ribeiro Notaro

Ouro Preto
Julho/2023

Introdução.....	3
Desenvolvimento.....	3
MMUSearchOnMemory().....	4
memoryCacheMapping().....	6
Testes.....	8
Método FIFO.....	9
Método LRU.....	9
Método Aleatório.....	9
Avaliação.....	10
Conclusão.....	10

Introdução

O Trabalho Prático II é levemente semelhante ao que foi proposto pelo primeiro no sentido de funcionamento do programa. Entretanto, aquele não possuía uma perspectiva de funcionamento de uma memória conhecida como “*cache*”, o que significa que a manipulação de memória era sempre feita diretamente à RAM. Neste, o fluxo de trabalho da máquina simulada é um pouco diferente principalmente por dois motivos: agora há memórias *caches* e o conceito de “bloco”, o qual está presente na memória RAM.

A memória *cache* é uma pequena e rápida memória localizada próxima ao processador que surgiu principalmente por causa do gargalo que se criou entre memória RAM e CPU durante os anos. Portanto, para se diminuir esse gargalo, introduziu-se essas pequenas memórias que, apesar de possuírem uma capacidade de armazenamento exponencialmente menor quando comparada à RAM, possuem uma alta velocidade e disponibilizam os dados mais recém utilizados pela própria CPU.

Desenvolvimento

No escopo geral, este trabalho se resume à função “MMUSearchOnMemory()”, cuja preocupação se limita à achar um certo bloco em uma das *caches*; em última instância, isto é, caso o bloco não esteja em qualquer uma das *caches*, ele é recuperado da memória RAM. Independentemente de onde foi achado, este bloco sempre será inserido na *cache* de menor nível. Se a cache de menor nível estiver cheia, um bloco seu é selecionado (e aqui entra a parte do mapeamento associativo) para que seja retirado e inserido na segunda *cache* de menor nível. Da mesma maneira, se a segunda *cache* de menor nível estiver cheia, um bloco seu é selecionado para que seja retirado e inserido na terceira *cache* de menor nível, e assim em diante até que não haja *caches* de níveis maiores onde um bloco possa ser inserido.

MMUSearchOnMemory()

É o principal método deste trabalho. Nele, está contida toda a lógica de busca de um bloco nas *caches* e o mapeamento realizado entre elas. Eis o método desenvolvido pelo grupo:

```
Line* MMUSearchOnMemorys(Address add, Machine* machine, WhereWasHit* whereWasHit) {
    // Strategy => write back
    MemoryBlock* RAM = machine->ram.blocks;
    Line* cache1 = machine->l1.lines;
    Line* cache2 = machine->l2.lines;
    Line* cache3 = machine->l3.lines;
    WhereWasHit cache_hit = RAMHit;

    int cost = 0;
    int cacheLine; // Line in cache L1 which must be returned

    // Is block in cache1?
    for(cacheLine = 0 ; cacheLine < machine->l1.size ; cacheLine++) {
        if(cache1[cacheLine].tag == add.block) {
            cache_hit = L1Hit;
            break;
        }
    }

    // If block is not in cache1, test if it is inside cache2.
    if(cache_hit != L1Hit) {
        for(cacheLine = 0 ; cacheLine < machine->l2.size ; cacheLine++) {
            if(cache2[cacheLine].tag == add.block) {
                cache_hit = L2Hit;
                break;
            }
        }
    }

    // If block is not in cache1, test if it is inside cache3.
    else if(cache_hit != L1Hit && cache_hit != L2Hit) {
        for(cacheLine = 0 ; cacheLine < machine->l3.size ; cacheLine++) {
            if(cache3[cacheLine].tag == add.block) {
                cache_hit = L3Hit;
                break;
            }
        }
    }

    if(cache_hit == L1Hit) {
        *whereWasHit = L1Hit;
        cost = COST_ACCESS_L1;
        // Uncomment to make the FIFO method.
        /*{
```

```

        rearrangeCacheLines(cache1, cacheLine, machine->l1.size);
        // Given the rearrangement, it is necessary to change where cacheLine
points to.
        cacheLine = machine->l1.size - 1;
    }*/
} else if(cache_hit == L2Hit) {
    *whereWasHit = L2Hit;
    cost = COST_ACCESS_L1 + COST_ACCESS_L2;
    {
        Line line = cache2[cacheLine];
        int newL1pos = memoryCacheMapping(machine, RAM, cache1, cache2, cache3);
        cache1[newL1pos] = line;
        // rearrangeCacheLines(cache2, cacheLine, machine->l2.size);
        cacheLine = newL1pos;
    }
} else if(cache_hit == L3Hit) {
    *whereWasHit = L3Hit;
    cost = COST_ACCESS_L1 + COST_ACCESS_L2 + COST_ACCESS_L3;
    {
        Line line = cache3[cacheLine];
        int newL1pos = memoryCacheMapping(machine, RAM, cache1, cache2, cache3);
        cache1[newL1pos] = line;
        // rearrangeCacheLines(cache3, cacheLine, machine->l3.size);
        cacheLine = newL1pos;
    }
} else if(cache_hit == RAMHit) {
    *whereWasHit = RAMHit;
    cost = COST_ACCESS_L1 + COST_ACCESS_L2 + COST_ACCESS_L3 + COST_ACCESS_RAM;
    {
        int newL1pos = memoryCacheMapping(machine, RAM, cache1, cache2, cache3);
        cache1[newL1pos].block = RAM[add.block];
        cache1[newL1pos].tag = add.block;
        cache1[newL1pos].updated = false;
        cacheLine = newL1pos;
    }
}

updateMachineInfos(machine, whereWasHit, cost);
return &(cache1[cacheLine]);
}

```

Com a exceção do cálculo do custo de acesso à uma das memórias e a atribuição à *flag* “whereWasHit”, o comportamento das condições é praticamente o mesmo:

1. Guarde a linha do bloco em questão em um registrador.

2. Faça o mapeamento das *caches* e retorne a linha de L1 em que será possível inserir o bloco armazenado no registrador.
3. Insira o bloco armazenado no registrador na linha retornada de L1.

Vale a observação de que os laços de repetição feitos anteriores às condições apontadas servem justamente para identificar em qual *cache* foi possível encontrar o bloco procurado. Se o bloco visado não estiver em nenhuma das *caches*, a *flag* “whereWasHit” terá o valor RAMHit, o que fará o programa cair na última condição.

Observe também que há partes comentadas que dizem respeito ao mapeamento utilizando a técnica *LRU* (*Last Recently Used*). Por motivos práticos e demonstrativos, essas linhas foram comentadas para que o processo de troca de mapeamento ficasse mais rápido, de modo que a apresentação também se tornasse mais dinâmica.

Muito ademais do método “MMUSearchOnMemory()”, há uma função mais complexa criada pelo grupo que representa o mapeamento, que é a “memoryCacheMapping”. Ela representa o ponto chave deste trabalho.

memoryCacheMapping()

Um adendo importante: foi explicitado no subtópico anterior que há linhas comentadas. Observe que o método de mapeamento vigente - *algo que é dedutível* - diz respeito ao LRU. Entretanto, veja também que as linhas comentadas representam o método aleatório de mapeamento.

```
int memoryCacheMapping(Machine* machine, MemoryBlock* RAM, Line* cache1, Line*
cache2, Line* cache3) {
    /* If it returns -1, there is not an unused line */
    int newLlpos = getUnusedLine(cache1, machine->l1.size);

    if(newLlpos == -1) {
        // Uncomment if it is desired to remove a line randomly.
        // removeLineRandomly(cache1, machine->l1.size, &newLlpos);
        Line removedLineL1 = removeLineFIFO(cache1, machine->l1.size);
```

```

/* New blocks in L1 are always inserted at the end, like a queue */
newL1pos = machine->l1.size - 1;

// if(!canOnlyReplaceBlock(cache1[newL1pos])) {
if(!canOnlyReplaceBlock(removedLineL1)) {
    /* If it returns -1, there is not an unused line */
    // int newL2pos = blockAlreadyExistsInCache(cache2, machine->l2.size,
    cache1[newL1pos].tag);
    int newL2pos = blockAlreadyExistsInCache(cache2, machine->l2.size,
    removedLineL1.tag);

    if(newL2pos == -1) {
        newL2pos = getUnusedLine(cache2, machine->l2.size);

        if(newL2pos == -1) {
            /* Gets the line which is going to be overwritten.*/
            // Uncomment if it is desired to remove a line randomly.
            // removeLineRandomly(cache2, machine->l2.size, &newL2pos);
            Line lineToOverwriteL2 = removeLineFIFO(cache2,
machine->l2.size);
            newL2pos = machine->l2.size - 1;

            // if(!canOnlyReplaceBlock(cache2[newL2pos])) {
            if(!canOnlyReplaceBlock(lineToOverwriteL2)) {
                /* If it returns -1, there is not an unused line */
                // int newL3pos = blockAlreadyExistsInCache(cache3,
machine->l3.size, cache2[newL2pos].tag);
                int newL3pos = blockAlreadyExistsInCache(cache3,
machine->l3.size, lineToOverwriteL2.tag);

                if(newL3pos == -1) {
                    newL3pos = getUnusedLine(cache3, machine->l3.size);

                    if(newL3pos == -1) {
                        // Uncomment if it is desired to remove a line
randomly.
                        // removeLineRandomly(cache3, machine->l3.size,
&newL3pos);

                        Line lineToOverwriteL3 = removeLineFIFO(cache3,
machine->l3.size);
                        newL3pos = machine->l3.size - 1;

                        // if(!canOnlyReplaceBlock(cache3[newL3pos]))
                        // RAM[cache3[newL3pos].tag] =
cache3[newL3pos].block;
                        if(!canOnlyReplaceBlock(lineToOverwriteL3))
                            RAM[lineToOverwriteL3.tag] =
lineToOverwriteL3.block;
                    }
                }
            }
        }
    }
}

```

```

        // cache3[newL3pos] = cache2[newL2pos];
        cache3[newL3pos] = lineToOverwriteL2;
    }
}

// cache2[newL2pos] = cache1[newL1pos];
cache2[newL2pos] = removedLineL1;
}
}

return newL1pos;
}

```

Embora seja uma função mais elaborada, o seu funcionamento não é nada complicado. Eis como funciona o fluxo de execução geral dela:

1. É possível simplesmente inserir na cache L1, evitando o mapeamento? Se sim, não entra em nenhuma condição e retorna a linha disponível em L1, caso contrário, utiliza-se da opção de mapeamento escolhida para remover a linha da cache L1 que saíra, fazendo com que a linha removida seja passada para a cache de nível superior, isto é, a L2.
2. Repita o processo 1 para todas as *caches* pré sequentes.

Testes

As instruções foram geradas pelo programa disponibilizado pelo docente. A quantidade de instruções geradas foi de 1000. O tamanho da memória RAM é de 1000; ou seja, ela possui 1000 blocos. O tamanho “exagerado” se deu por conta das memórias *caches*, será observado que na tabela de teste montada, o tamanho das *caches* às vezes chegava em casas centenárias.

Para fins de simplificação, as instruções dizem tão somente sobre as operações básicas, ou seja: adição, subtração e atribuição.

Método FIFO

	Cache			Taxa Hit Cache (%)			Quantidade Hit RAM	Custo de execução
	L1	L2	L3	L1	L2	L3		
M1	8	16	32	0,1	34	0	1676	1871719
M2	32	64	128	22	17	0	475	530799
M3	16	64	256	61	29	0	724	809149
M4	8	32	128	0,1	37	0	1603	1791419
M5	16	32	64	60	28,5	0	728	813549

Método LRU

	Cache			Taxa Hit Cache (%)			Quantidade Hit RAM	Custo de execução
	L1	L2	L3	L1	L2	L3		
M1	8	16	32	0,1	37,7	0	1590	1777119
M2	32	64	128	85	3	0	367	410049
M3	16	64	256	64,8	27	0	656	733159
M4	8	32	128	0,1	37,9	0	1586	1772719
M5	16	32	64	64,8	26,6	0	660	737559

Método Aleatório

	Cache			Taxa Hit Cache (%)			Quantidade Hit RAM	Custo de execução
	L1	L2	L3	L1	L2	L3		
M1	8	16	32	19,7	31	0	1413	1577399
M2	32	64	128	77,9	17,7	0	464	518599
M3	16	64	256	63,7	28,6	0	663	741149

M4	8	32	128	18	34	0	1373	1533769
M5	16	32	64	61,9	26	0	718	802089

Avaliação

Ao comparar a tabela do FIFO com a da LRU, fica claro que o método LRU se sobressai. A explicação para isso é que o último sempre faz com que os blocos utilizados nas *caches* vão para a última posição da fila, fazendo com que os blocos mais recém utilizados sejam os últimos a serem removidos, enquanto no método FIFO isso não ocorre. Neste último, mesmo que um bloco tenha sido recém utilizado, ele continua na sua posição na fila.

O que é realmente interessante e bastante inusitado é a tabela do método Aleatório, quando comparada à do LRU. Por incrível que pareça, o método aleatório se encontrará em situações onde irá obter vantagem em cima do LRU. Pelo levantamento de dados realizado pelas tabelas, a conclusão que o grupo chegou foi a seguinte: quando a cache L1 possuir um tamanho igual ou menor que 8 linhas, o método aleatório será mais interessante, caso contrário, ele passará a ser mais insustentável.

A parte do código desenvolvido pelo grupo que possui mais iterações diz respeito àquela em que há a tentativa de se realizar um *cache hit*, isto é, tentar achar um certo bloco que é desejado dentro das *caches*. De maneira mais rígida, percebe-se que a ordem de complexidade do código é $O(3n)$, entretanto, já que o expoente dominante é 1, pode-se dizer que a ordem de complexidade do código é $O(n)$.

Conclusão

O Trabalho Prático I oferecia a oportunidade de entendimento de como funcionam as instruções, o que foi algo muito interessante. Com este Trabalho Prático II, o conceito das instruções não deixou de estar presente, embora não tenha sido o enfoque. Devido à proposta deste trabalho prático, foi possível

entender o funcionamento de uma memória *cache* e o fluxo de trabalho entre as *caches* e a memória RAM.

Além das importâncias apontadas, há o conceito principal do trabalho: o mapeamento associativo. Entendê-lo e saber diferenciá-lo do mapeamento direto é uma capacidade que este trabalho prático proporcionou.

Por fim, o grupo responsável teve como maior dificuldade a criação do fluxo de verificações que o programa possui, que, no escopo ampliado, diz respeito à transferência de blocos entre as *caches* e quando era ou não possível realizar essa transferência e em quais circunstâncias.